

ONBRD-09: Setting Up Alerts

Series: ONBRD — Dynatrace Onboarding | **Notebook:** 9 of 10 | **Created:** December 2025 | **Last Updated:** 09/24/2026

Getting Notified When Things Go Wrong

Dynatrace's DAVIS AI automatically detects problems, but you need to configure where those alerts go. This notebook covers the Workflows app for modern alerting and notification routing.

Table of Contents

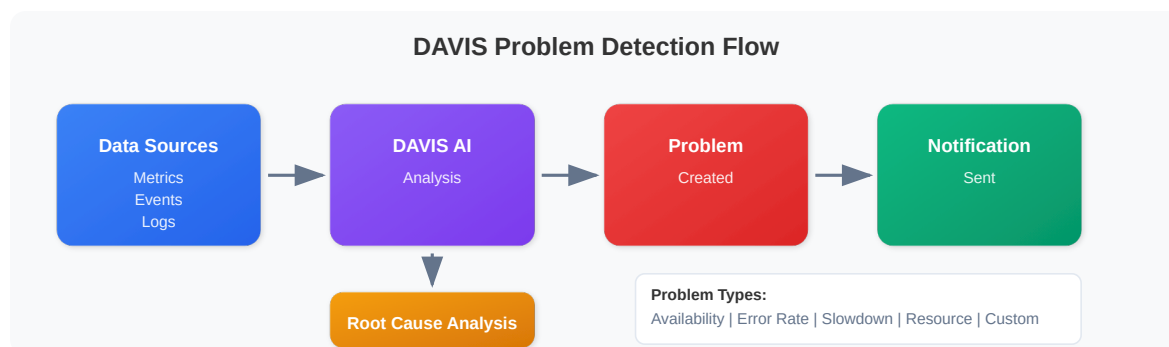
1. [How DAVIS Problem Detection Works](#)
 2. [Modern Alerting with Workflows](#)
 3. [Creating Your First Workflow](#)
 4. [Notification Actions](#)
 5. [Routing Alerts to Teams](#)
 6. [Custom Metric Alerts](#)
 7. [Next Steps](#)
-

Prerequisites

- Configurator or Admin access
- DQL fundamentals (ONBRD-08)
- Access to notification target (email, Slack, PagerDuty, etc.)

1. How DAVIS Problem Detection Works

DAVIS AI continuously monitors your environment and creates **problems** when anomalies are detected:



Problem Types

Type	Trigger	Example
Availability	Service/process unavailable	Database crashed
Error rate	Error rate increases	500 errors spike
Slowdown	Response time degradation	Latency increase
Resource	CPU, memory, disk issues	Disk full
Custom	Metric thresholds breached	Custom alert

2. Modern Alerting with Workflows

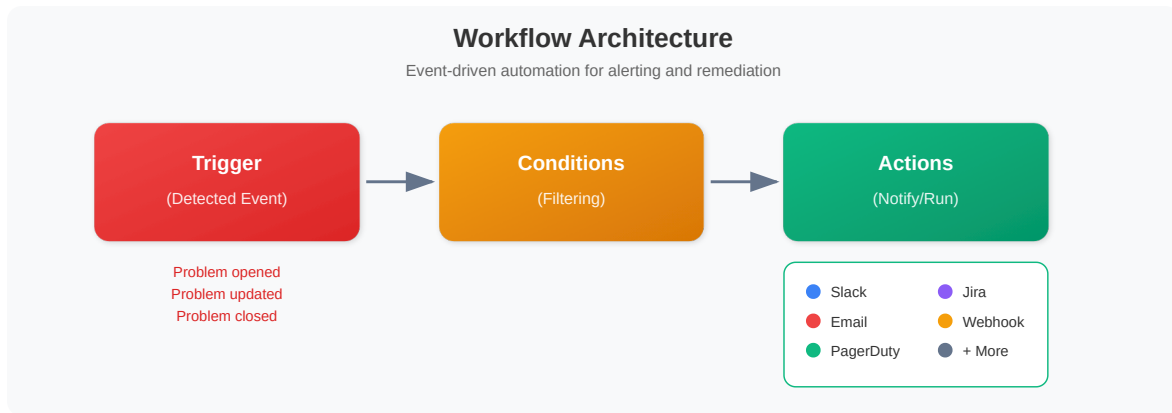
The Workflows app is the modern platform's approach to alerting and automation.

Location: Automate → Workflows

What are Workflows?

Workflows are event-driven automations that can:

- React to DAVIS problems
- Send notifications to various channels
- Execute remediation actions
- Run on schedules



Workflow vs Legacy Alerting Profiles

Feature	Workflows	Alerting Profiles (Legacy)
Trigger types	Events, schedules, manual	detected problems only
Filtering	DQL matchers on the problem	Rule-based
Actions	20+ built-in actions	Fixed notifications
Automation	Full automation capability	Notification only
Modern platform	Recommended	Dynatrace Classic — supported, not deprecated

On the status of alerting profiles. No Dynatrace page announces a deprecation or an end-of-life date for alerting profiles. The alerting-profiles page states *"Problem notification is a Dynatrace Classic concept."* and steers new work elsewhere: *"Use simple workflows to send notifications about problems."* Treat that as *build new alerting on workflows*, not as *your existing profiles are about to stop working*.

One caveat does bite: an alerting profile scoped by a **Management Zone** is only as durable as that zone. The MZ filter has no successor in the alerting model, so teams retiring Management Zones must rebuild those profiles as problem-triggered workflows first. See MZ2POL-01 §5.

Sources: [Problem alerting profiles \(DT docs\)](#) — the two quotes above, [Upgrade guide: alert notifications \(DT docs\)](#) — *"A workflow's Problem trigger filters problems directly with DQL matchers on the problem."*

3. Creating Your First Workflow

Step 1: Open the Workflows App

Location: Automate → Workflows → Create workflow

Step 2: Configure the Trigger

1. Click "Add trigger"
2. Select "detected problem" trigger
3. Configure trigger options:
 - **Problem opens** - When a new problem is detected
 - **Problem updates** - When problem details change
 - **Problem closes** - When a problem is resolved

Step 3: Add Conditions (Optional)

Filter which problems trigger the workflow. Use the trigger's own options first (problem state, event category, severity, affected-entity tags). Anything more goes in **Additional custom filter query**, which takes a *"DQL matcher expression to further refine which problems start the trigger"* — DQL matcher syntax on the problem record, not JavaScript ([Event triggers for workflows \(DT docs\)](#)):

```
// Example: availability problems only
event.category == "AVAILABILITY"
```

To limit it to production, use the trigger's **Affected entities** tag filter (for example an `environment:production` tag). Do not match on entity IDs: an ID such as `CLOUD_APPLICATION-EADC52AF343668DE` carries no environment or application name — none of 3,210 problems on a validation tenant had `prod` in an affected-entity ID (09/24/2026).

Step 4: Add Actions

1. Click "+" to add an action
2. Select action type (Slack, Email, PagerDuty, etc.)
3. Configure the action parameters

Basic Workflow Example

```
Trigger: detected problem opens
Condition (custom filter): event.category == "ERROR"
Action: Send Slack message to #alerts channel
```

Workflow Settings

Setting	Description
Name	Descriptive workflow name
Description	What this workflow does
Owner	User or service account
State	Enabled/Disabled

4. Notification Actions

Workflows support multiple notification channels.

Built-in Notification Actions

Action	Use Case
Send email	Basic notifications
Send Slack message	Team channels
Send Microsoft Teams message	Team channels
Create PagerDuty incident	On-call paging
Create ServiceNow incident	Incident tickets
Create Jira issue	Issue tracking
Send webhook	Custom integrations
Create OpsGenie alert	Alert management

Setting Up Slack Notifications

1. First, connect Slack to Dynatrace:
 - Go to Settings → Integration → Slack
 - Follow the OAuth flow to connect your workspace
2. In your workflow, add a Slack action:
 - Select channel
 - Configure message template
 - Use variables for dynamic content

Message Templates

Use Jinja2 templates for dynamic messages:

```
*Problem Detected*
*Title:* {{ event()["event.name"] }}
*Category:* {{ event()["event.category"] }}
*Status:* {{ event()["event.status"] }}
*Link:* {{ problem_link() }}
```

Template fields come from the problem record. The Problem trigger's `event()` is the `dt.davis.problems` record — run `fetch dt.davis.problems, from:-24h | limit 1` to see every field a template can read. It has no `title` or `problem_url` field (0 of 3,209 problem records on a validation tenant, 09/24/2026): the title is `event.name`, the kind of problem is `event.category`, and the link is `{{ problem_link() }}`, which *"evaluates correctly in workflows with Davis problem event triggers only."* ([Jinja expressions for Workflows \(DT docs\)](#), [Event triggers for workflows \(DT docs\)](#))

Setting Up Email Notifications

1. Add "Send email" action to workflow
2. Configure:
 - Recipients (to, cc, bcc)
 - Subject line (can use templates)
 - Body content (HTML or plain text)

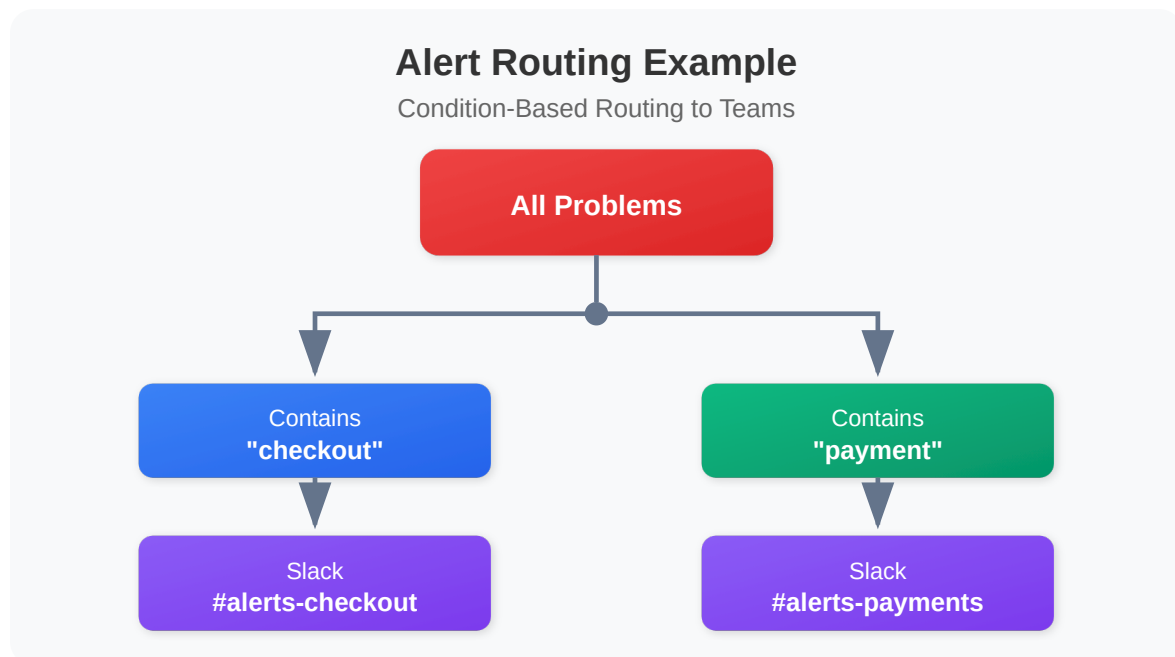
Setting Up PagerDuty

1. Go to Settings → Integration → PagerDuty
2. Configure your PagerDuty integration key
3. Add "Create PagerDuty incident" action to workflow
4. Map severity levels appropriately

5. Routing Alerts to Teams

Use workflow conditions to route alerts to the right teams.

Strategy: Condition-Based Routing



Routing by Entity Name

Entity IDs never contain names, so match on `affected_entity_names`. In a DQL matcher, `matchesValue` "works with multi-value attributes (matching any value), and supports wildcards" ([DQL matcher in OpenPipeline \(DT docs\)](#)). On a validation tenant (09/24/2026) `matchesValue(affected_entity_names, "*payment*")` matched 268 problems; searching the entity IDs for `payment` matched none.

```
// Additional custom filter query – route checkout team alerts
matchesValue(affected_entity_names, "*checkout*")
```

Routing by Problem Category

```
// Route availability issues to SRE
event.category == "AVAILABILITY"

// Route performance issues to app team
event.category == "SLOWDOWN"
```

Example Multi-Team Setup

Team	Workflow	Condition	Channel
Checkout	checkout-alerts	Entity contains "checkout"	Slack #alerts-checkout
Payments	payments-alerts	Entity contains "payment"	PagerDuty Payments
Platform	critical-alerts	Category == "AVAILABILITY"	PagerDuty Platform

Creating Team-Specific Workflows

1. Create one workflow per team/routing need
2. Use conditions to filter problems
3. Send to appropriate channel
4. Include relevant context in message

6. Custom Metric Alerts

Create alerts based on specific metric thresholds using the Analyzer in Workflows.

When to Use Custom Metric Alerts

Scenario	Configuration
Disk > 90%	Static threshold

Scenario	Configuration
Queue depth spike	Deviation from baseline
Business metric	Custom metric threshold
SLO breach	SLO burn rate

Creating a Metric-Based Workflow

1. Go to Automate → Workflows
2. Create new workflow
3. Add trigger: "detected problem"
4. Add condition to filter for metric events
5. Add notification action

Using Analyzers

Analyzers can detect anomalies in metrics:

- **Static threshold** - Alert when value exceeds X
- **Auto-adaptive baseline** - Alert on deviations from normal
- **Seasonal patterns** - Account for time-based variations

Example: High CPU Alert

1. Create workflow with detected problem trigger
2. Add condition (Additional custom filter query):

```
text event.category == "RESOURCE_CONTENTION" and matchesPhrase(event.name, "CPU")
```

The category is `RESOURCE_CONTENTION` — there is no `RESOURCE` category — and the problem title is `event.name`. On a validation tenant (7 days to 09/24/2026) this matched 1,385 of 1,464 resource-contention problems; `event.category == "RESOURCE"` matched none.

3. Add Slack notification action

```
// Recent problems
fetch dt.davis.problems, from: now() - 24h
| fields timestamp, display_id, event.name, event.status,
affected_entity_types
| sort timestamp desc
| limit 20
```

```
// Problem count by status
fetch dt.davis.problems, from: now() - 7d
| summarize count = count(), by: {event.status}
| sort count desc
```

```
// Problems by day
fetch dt.davis.problems, from: now() - 7d
| fieldsAdd day = bin(timestamp, 1d)
| summarize problem_count = count(), by: {day}
| sort day desc
```

```
// Active problems right now
fetch dt.davis.problems, from: now() - 30d
| filter event.status == "ACTIVE"
| fields timestamp, display_id, event.name, affected_entity_types
| sort timestamp desc
```

```
// Problem duration analysis
fetch dt.davis.problems, from: now() - 7d
| filter event.status == "CLOSED"
| filter isNotNull(event.start) and isNotNull(event.end)
| fieldsAdd duration_minutes = (event.end - event.start) / 1m
| summarize {
  avg_duration = avg(duration_minutes),
  max_duration = max(duration_minutes),
  problem_count = count()
}
```

Alert Testing Checklist

After configuring workflows:

1. **Test notification delivery** - Use workflow test feature
2. **Verify routing** - Confirm correct channels receive alerts
3. **Check formatting** - Review message content

4. **Validate conditions** - Ensure filters work as expected
5. **Test resolution** - Confirm close notifications work

Workflow Execution History

View workflow runs in:

- Automate → Workflows → Select workflow → Executions

Check for:

- Successful runs
- Failed runs with error details
- Action outputs

7. Next Steps

With alerting configured:

1. **ONBRD-10: Building Dashboards** — Visualize your data
2. Fine-tune workflow conditions based on alert volume
3. Set up escalation paths with multiple workflows
4. Document on-call procedures

Where to Go Deeper

- **AIOPS series** (8 notebooks) — Davis AI in depth: Causal/Predictive/Generative, anomaly detection mechanisms (static / auto-adaptive / seasonal / multi-dimensional baseline / novelty/forecast), Davis problems & RCA, Davis CoPilot / Dynatrace Assist, AI models, integrations & agentic workflows
- **WFLOW series** (12 notebooks) — Workflows depth: triggers, actions, AI tasks, scheduled workflows, MCP server integration

Alerting Checklist

- ☐ Slack/Email integration configured
- ☐ First workflow created and tested
- ☐ Team-specific workflows configured
- ☐ Test notifications sent successfully

- [] Critical path to on-call established
 - [] Escalation procedures documented
 - [] Davis sensitivity defaults reviewed
-

Summary

In this notebook, you learned:

- How DAVIS problem detection works
 - How to create workflows for alerting
 - How to configure notification actions
 - How to route alerts to teams using conditions
 - How to create custom metric alerts
 - How to monitor workflow effectiveness
 - That `dt.davis.problems` uses `event.status` / `event.end` fields, and durations should use `/ 1m` (duration arithmetic), not `/ 1m` (nanosecond constants)
-

References

- [Root cause analysis \(DT docs\)](#)
 - [Workflows](#)
 - [Workflow Actions](#)
 - [Slack Integration](#)
 - [PagerDuty Integration](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.